

Module 8.4 - Advanced Component Techniques

Learning Objectives:

- Master content projection with ng-content
 - Implement single and multi-slot content projection
 - Load components dynamically at runtime
 - Use ViewContainerRef for dynamic component creation
 - Apply advanced component composition patterns
-

Table of Contents

1. [Content Projection with ng-content](#)
 2. [Multi-Slot Content Projection](#)
 3. [Conditional Content Projection](#)
 4. [Dynamic Component Loading](#)
 5. [ViewContainerRef and ComponentFactory](#)
 6. [Advanced Patterns and Use Cases](#)
 7. [Practical Examples](#)
 8. [Summary and Key Takeaways](#)
-

1. Content Projection with ng-content

1.1 What is Content Projection?

Content projection (also called transclusion) is a pattern where you pass content from a parent component to a child component, and the child component displays that content in a designated spot within its template.

Why Use Content Projection?

- Create reusable wrapper components (cards, modals, tabs)
- Build flexible UI components that can accept any content
- Separate container logic from content
- Enable component composition and customization

1.2 Basic Single-Slot Projection

The simplest form of content projection uses a single `<ng-content>` element.

Child Component (Card):

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <div class="card-header">
        <h3>Card Title</h3>
      </div>
      <div class="card-body">
        <ng-content></ng-content> <!-- Content goes here -->
      </div>
    </div>
  `,
  styles: [`
    .card {
      border: 1px solid #ddd;
      border-radius: 8px;
      padding: 16px;
      margin: 10px;
      box-shadow: 0 2px 4px rgba(0,0,0,0.1);
    }
    .card-header {
      border-bottom: 1px solid #eee;
      margin-bottom: 12px;
      padding-bottom: 8px;
    }
  `]
})
```

```
`]
})
export class CardComponent {}
```

Parent Component Usage:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-dashboard',
  template: `
    <h2>Dashboard</h2>

    <app-card>
      <p>This content is projected into the card!</p>
      <button>Click Me</button>
    </app-card>

    <app-card>
      <ul>
        <li>Item 1</li>
        <li>Item 2</li>
        <li>Item 3</li>
      </ul>
    </app-card>

    <app-card>
      
      <p>Image caption</p>
    </app-card>
  `
})
export class DashboardComponent {}
```

💡 **Key Point:** Everything between the `<app-card>` tags is projected into the `<ng-content>` slot.

1.3 How Content Projection Works

```
Parent Component
├─ <app-card>
│   └─ <p>Content</p> ← This content...
│       └─ <button>...</button>
```

```
└─ </app-card>
```

Child Component (CardComponent)

```
├─ <div class="card">
│   └─ <div class="card-body">
│       └─ <ng-content></ng-content> ← ...goes here
│   └─ </div>
└─ </div>
```

1.4 Practical Example: Alert Component

```
import { Component } from '@angular/core';
```

```
@Component({
  selector: 'app-alert',
  template: `
    <div class="alert">
      <ng-content></ng-content>
    </div>
  `,
  styles: [
    `.alert {
      padding: 12px;
      border: 1px solid #ddd;
      border-radius: 4px;
      background: #f8f9fa;
    }`
  ]
})
export class AlertComponent {}
```

Usage:

```
<app-alert>
  <strong>Success!</strong> Your changes have been saved.
</app-alert>

<app-alert>
  <p>Any content can go here!</p>
</app-alert>
```

2. Multi-Slot Content Projection

2.1 Named Slots with select Attribute

You can project content into multiple specific locations using the `select` attribute on `<ng-content>`.

Child Component (Dialog):

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-dialog',
  template: `
    <div class="dialog">
      <div class="dialog-header">
        <ng-content select="[header]"></ng-content>
      </div>
      <div class="dialog-body">
        <ng-content></ng-content>
      </div>
      <div class="dialog-footer">
        <ng-content select="[footer]"></ng-content>
      </div>
    </div>
  `,
  styles: [`
    .dialog { border: 1px solid #ddd; border-radius: 4px; }
    .dialog-header { padding: 12px; border-bottom: 1px solid #eee; }
    .dialog-body { padding: 16px; }
    .dialog-footer { padding: 12px; border-top: 1px solid #eee; }
  `]
})
export class DialogComponent {}
```

Parent Component Usage:

```
<app-dialog>
  <h3 header>Confirm Action</h3>
  <p>Are you sure?</p>
  <div footer>
    <button>Cancel</button>
  </div>
</app-dialog>
```

```

    <button>OK</button>
  </div>
</app-dialog>

```

2.2 Selection Strategies

Selector Type	Syntax	Example
Attribute	<code>select="[attr]"</code>	<code><ng-content select="[header]"></code>
Element	<code>select="element"</code>	<code><ng-content select="header"></code>
Class	<code>select=".class"</code>	<code><ng-content select=".title"></code>
CSS Selector	<code>select="any-css"</code>	<code><ng-content select="h2.title"></code>
Default	(no select)	<code><ng-content></code> (catches everything else)

2.3 Example: Panel Component

```

import { Component } from '@angular/core';

@Component({
  selector: 'app-panel',
  template: `
    <div class="panel">
      <h3><ng-content select=".title"></ng-content></h3>
      <div><ng-content></ng-content></div>
    </div>
  `,
  styles: [`
    .panel { border: 1px solid #ddd; padding: 16px; }
  `]
})
export class PanelComponent {}

```

Usage:

```
<app-panel>
  <span class="title">Panel Title</span>
  <p>Main content goes here</p>
</app-panel>
```

2.4 Default Slot (Fallback Content)

An `<ng-content>` without a `select` attribute captures any content that doesn't match other selectors.

```
@Component({
  selector: 'app-panel',
  template: `
    <div class="panel">
      <h3><ng-content select=".title"></ng-content></h3>
      <div class="panel-body">
        <ng-content></ng-content> <!-- Default slot -->
      </div>
      <footer><ng-content select="footer"></ng-content></footer>
    </div>
  `
})
export class PanelComponent {}
```

Usage:

```
<app-panel>
  <span class="title">Panel Title</span>

  <!-- These go to default slot -->
  <p>Main content paragraph 1</p>
  <p>Main content paragraph 2</p>

  <footer>Panel Footer</footer>
</app-panel>
```

3. Conditional Content Projection

3.1 Using ngProjectAs

The `ngProjectAs` attribute allows you to specify which slot should receive content, even when wrapped in structural directives.

```
@Component({
  selector: 'app-card-advanced',
  template: `
    <div class="card">
      <div class="card-header">
        <ng-content select="[card-title]"></ng-content>
      </div>
      <div class="card-body">
        <ng-content></ng-content>
      </div>
    </div>
  `
})
export class CardAdvancedComponent {}
```

Without `ngProjectAs` (doesn't work):

```
<app-card-advanced>
  <!-- *ngIf wraps in ng-template, breaks projection -->
  <h3 *ngIf="showTitle" card-title>{{ title }}</h3>
  <p>Body content</p>
</app-card-advanced>
```

With `ngProjectAs` (works):

```
<app-card-advanced>
  <ng-container *ngIf="showTitle" ngProjectAs="[card-title]">
    <h3>{{ title }}</h3>
  </ng-container>
  <p>Body content</p>
</app-card-advanced>
```


3.2 Checking for Projected Content

You can check if content was projected using `@ContentChild` or `@ContentChildren`.

```
import { Component, ContentChild, ElementRef, AfterContentInit } from '@angular/core';

@Component({
  selector: 'app-expandable',
  template: `
    <div class="expandable">
      <div class="header" (click)="toggle()">
        <ng-content select="[header]"></ng-content>
        <span class="icon">{{ isExpanded ? '▼' : '►' }}</span>
      </div>

      <div class="content" *ngIf="isExpanded">
        <ng-content></ng-content>
      </div>

      <div class="footer" *ngIf="hasFooter">
        <ng-content select="[footer]"></ng-content>
      </div>
    </div>
  `
})
export class ExpandableComponent implements AfterContentInit {
  @ContentChild('footer', { read: ElementRef }) footerContent?: ElementRef;

  isExpanded = false;
  hasFooter = false;

  ngAfterContentInit(): void {
    this.hasFooter = !!this.footerContent;
  }

  toggle(): void {
    this.isExpanded = !this.isExpanded;
  }
}
```

4. Dynamic Component Loading

4.1 What is Dynamic Component Loading?

Dynamic component loading allows you to create and insert components at runtime, rather than declaring them in templates at compile time.

Use Cases:

- Loading components based on user actions or server responses
- Plugin architectures
- Dynamic forms
- Ad/widget systems
- Modal dialogs
- Notifications/toasts

4.2 Modern Approach (Angular 13+): ViewContainerRef

In modern Angular, we use `ViewContainerRef.createComponent()` directly without `ComponentFactoryResolver`.

Component to Load Dynamically:

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-message',
  template: `
    <div class="message">
      <p>{{ text }}</p>
      <button (click)="close()">Close</button>
    </div>
  `,
  styles: [
    `.message { padding: 16px; border: 1px solid #ddd; margin: 8px; }`
  ]
})
export class MessageComponent {
```

```
@Input() text: string = '';
@Input() close: () => void = () => {};
}
```

Host Component:

```
import { Component, ViewChild, ViewContainerRef } from '@angular/core';
import { MessageComponent } from './message.component';

@Component({
  selector: 'app-host',
  template: `
    <button (click)="showMessage()">Show Message</button>
    <div #container></div>
  `
})
export class HostComponent {
  @ViewChild('container', { read: ViewContainerRef })
  container!: ViewContainerRef;

  showMessage(): void {
    // Create component dynamically
    const componentRef = this.container.createComponent(MessageComponent);

    // Set inputs
    componentRef.instance.text = 'Hello from dynamic component!';
    componentRef.instance.close = () => componentRef.destroy();
  }
}
```

4.3 Using ng-template as Insertion Point

```
import { Component, ViewChild, ViewContainerRef, TemplateRef } from '@angular/core';

@Component({
  selector: 'app-dynamic-host',
  template: `
    <div>
      <button (click)="loadComponent()">Load Component</button>
      <button (click)="clearAll()">Clear All</button>

      <!-- Insertion point using ng-template -->
    </div>
  `
})
export class DynamicHostComponent {
  @ViewChild(TemplateRef)
  templateRef!: TemplateRef<any>;

  loadComponent(): void {
    // ...
  }

  clearAll(): void {
    // ...
  }
}
```

```
        <ng-template #dynamicContainer></ng-template>
    </div>
    `
  })
  export class DynamicHostComponent {
    @ViewChild('dynamicContainer', { read: ViewContainerRef })
    container!: ViewContainerRef;

    loadComponent(): void {
      const componentRef = this.container.createComponent(MessageComponent);
      componentRef.instance.text = 'Dynamic Message!';
    }

    clearAll(): void {
      this.container.clear();
    }
  }
}
```

4.4 Passing Data to Dynamic Components

Method 1: Setting Inputs Directly

```
const componentRef = this.container.createComponent(MyComponent);
componentRef.instance.title = 'Hello';
componentRef.instance.count = 42;
```

Method 2: Using setInput() (Angular 14+)

```
const componentRef = this.container.createComponent(MyComponent);
componentRef.setInput('title', 'Hello');
componentRef.setInput('count', 42);
```

Method 3: Subscribing to Outputs

```
import { Component, EventEmitter, Output } from '@angular/core';

@Component({
  selector: 'app-user-form',
  template: `
    <form>
```

```

    <input [(ngModel)]="name" placeholder="Name">
    <button (click)="submit()">Submit</button>
  </form>
  ,
})
export class UserFormComponent {
  name = '';
  @Output() formSubmit = new EventEmitter<string>();

  submit(): void {
    this.formSubmit.emit(this.name);
  }
}

// In host component
const componentRef = this.container.createComponent(UserFormComponent);
componentRef.instance.formSubmit.subscribe((name: string) => {
  console.log('Form submitted with:', name);
  componentRef.destroy();
});

```

5. ViewContainerRef and ComponentFactory

5.1 Understanding ViewContainerRef

ViewContainerRef represents a container where one or more views can be attached. It provides methods to create, move, and remove views dynamically.

Key Methods:

Method	Description
<code>createComponent()</code>	Creates and inserts a component
<code>createEmbeddedView()</code>	Creates a view from a template
<code>clear()</code>	Removes all views
<code>insert()</code>	Inserts a view at a specific index

Method	Description
<code>move()</code>	Moves a view to a different index
<code>remove()</code>	Removes a view at a specific index
<code>get()</code>	Retrieves a view by index
<code>length</code>	Number of views in the container

5.2 Creating Views from Templates

```
import { Component, ViewChild, ViewContainerRef, TemplateRef } from '@angular/core';

@Component({
  selector: 'app-template-loader',
  template: `
    <button (click)="loadTemplate()">Load Template</button>
    <button (click)="clear()">Clear</button>

    <div #container></div>

    <ng-template #myTemplate let-name="name" let-age="age">
      <div class="user-card">
        <h3>{{ name }}</h3>
        <p>Age: {{ age }}</p>
      </div>
    </ng-template>
  `
})
export class TemplateLoaderComponent {
  @ViewChild('container', { read: ViewContainerRef })
  container!: ViewContainerRef;

  @ViewChild('myTemplate', { read: TemplateRef })
  template!: TemplateRef<any>;

  loadTemplate(): void {
    // Create embedded view with context
    const view = this.container.createEmbeddedView(this.template, {
      name: 'John Doe',
      age: 30
    });
  }
}
```

```
// Optional: Access view data
console.log('View created:', view);
}

clear(): void {
  this.container.clear();
}
}
```

5.3 Multiple Dynamic Components

```
import { Component, ViewChild, ViewContainerRef, Type } from '@angular/core';

@Component({
  selector: 'app-multi-loader',
  template: `
    <div>
      <button (click)="loadComponent(componentA)">Load Component A</button>
      <button (click)="loadComponent(componentB)">Load Component B</button>
      <button (click)="clearAll()">Clear All</button>
      <p>Loaded components: {{ container?.length || 0 }}</p>

      <div #container></div>
    </div>
  `,
})
export class MultiLoaderComponent {
  @ViewChild('container', { read: ViewContainerRef })
  container!: ViewContainerRef;

  componentA = ComponentA;
  componentB = ComponentB;

  loadComponent(componentType: Type<any>): void {
    const componentRef = this.container.createComponent(componentType);
    console.log('Loaded:', componentRef.instance.constructor.name);
  }

  clearAll(): void {
    this.container.clear();
  }
}

@Component({
```

```
selector: 'app-component-a',
template: `<div class="box-a">Component A</div>`,
styles: [`.box-a {
  padding: 20px;
  background: lightblue;
  margin: 10px;
}`]
})
export class ComponentA {}

@Component({
  selector: 'app-component-b',
  template: `<div class="box-b">Component B</div>`,
  styles: [`.box-b {
    padding: 20px;
    background: lightgreen;
    margin: 10px;
 }`]
})
export class ComponentB {}
```

5.4 Legacy Approach: ComponentFactoryResolver (Pre-Angular 13)

 **Note:** ComponentFactoryResolver is deprecated in Angular 13+. Use `ViewContainerRef.createComponent()` directly instead.

```
// Legacy approach (deprecated)
import { ComponentFactoryResolver } from '@angular/core';

constructor(
  private componentFactoryResolver: ComponentFactoryResolver
) {}

loadComponent(): void {
  const factory = this.componentFactoryResolver
    .resolveComponentFactory(MyComponent);
  const componentRef = this.container.createComponent(factory);
}

// Modern approach (Angular 13+)
```



```
loadComponent(): void {  
  const componentRef = this.container.createComponent(MyComponent);  
}
```

6. Advanced Patterns and Use Cases

6.1 Modal Service with Dynamic Components

```
import { Injectable, ViewContainerRef, ComponentRef, Type } from '@angular/core';  
  
@Injectable({  
  providedIn: 'root'  
})  
export class ModalService {  
  private containerRef?: ViewContainerRef;  
  private modalRef?: ComponentRef<any>;  
  
  setRootViewContainerRef(viewContainerRef: ViewContainerRef): void {  
    this.containerRef = viewContainerRef;  
  }  
  
  open<T>(component: Type<T>, inputs?: Partial<T>): ComponentRef<T> {  
    if (!this.containerRef) {  
      throw new Error('ViewContainerRef not initialized');  
    }  
  
    // Create modal component  
    this.modalRef = this.containerRef.createComponent(component);  
  
    // Set inputs if provided  
    if (inputs) {  
      Object.keys(inputs).forEach(key => {  
        (this.modalRef!.instance as any)[key] = (inputs as any)[key];  
      });  
    }  
  
    return this.modalRef as ComponentRef<T>;  
  }  
  
  close(): void {  
    if (this.modalRef) {
```

```
    this.modalRef.destroy();  
    this.modalRef = undefined;  
  }  
}  
}
```

Modal Component:

```
import { Component, Input, Output, EventEmitter } from '@angular/core';  
  
@Component({  
  selector: 'app-confirm-modal',  
  template: `  
    <div class="modal-backdrop">  
      <div class="modal">  
        <h2>{{ title }}</h2>  
        <p>{{ message }}</p>  
        <div class="buttons">  
          <button (click)="onCancel()">Cancel</button>  
          <button (click)="onConfirm()" class="primary">Confirm</button>  
        </div>  
      </div>  
    </div>  
  `,  
  styles: [`  
    .modal-backdrop {  
      position: fixed;  
      top: 0;  
      left: 0;  
      right: 0;  
      bottom: 0;  
      background: rgba(0, 0, 0, 0.5);  
      display: flex;  
      align-items: center;  
      justify-content: center;  
    }  
    .modal {  
      background: white;  
      padding: 30px;  
      border-radius: 8px;  
      min-width: 400px;  
    }  
    .buttons {  
      display: flex;  
      gap: 10px;  
    }  
  ]  
})
```

```

    justify-content: flex-end;
    margin-top: 20px;
  }
`]
})
export class ConfirmModalComponent {
  @Input() title = 'Confirm';
  @Input() message = 'Are you sure?';
  @Output() confirm = new EventEmitter<void>();
  @Output() cancel = new EventEmitter<void>();

  onConfirm(): void {
    this.confirm.emit();
  }

  onCancel(): void {
    this.cancel.emit();
  }
}

```

Usage in Component:

```

import { Component, ViewChild, ViewContainerRef, OnInit } from '@angular/core';
import { ModalService } from '../modal.service';
import { ConfirmModalComponent } from '../confirm-modal.component';

@Component({
  selector: 'app-root',
  template: `
    <button (click)="openModal()">Open Modal</button>
    <ng-template #modalContainer></ng-template>
  `
})
export class AppComponent implements OnInit {
  @ViewChild('modalContainer', { read: ViewContainerRef })
  modalContainer!: ViewContainerRef;

  constructor(private modalService: ModalService) {}

  ngOnInit(): void {
    this.modalService.setRootViewContainerRef(this.modalContainer);
  }

  openModal(): void {
    const modalRef = this.modalService.open(ConfirmModalComponent, {

```

```
    title: 'Delete Item',
    message: 'Are you sure you want to delete this item?'
  });

  modalRef.instance.confirm.subscribe(() => {
    console.log('Confirmed!');
    this.modalService.close();
  });

  modalRef.instance.cancel.subscribe(() => {
    console.log('Cancelled!');
    this.modalService.close();
  });
}
}
```

6.2 Loading Multiple Components

Concept: Create multiple components in a loop.

```
@Component({
  selector: 'app-list',
  template: `
    <button (click)="addItem()">Add Item</button>
    <ng-template #list></ng-template>
  `
})
export class ListComponent {
  @ViewChild('list', { read: ViewContainerRef })
  container!: ViewContainerRef;

  addItem(): void {
    const ref = this.container.createComponent(ItemComponent);
    ref.instance.text = `Item ${this.container.length}`;
  }
}

@Component({
  selector: 'app-item',
  template: `<div>{{ text }}</div>`
})
```

```
export class ItemComponent {  
  text = '';  
}
```

6.3 Destroying Components

Concept: Store references to destroy components later.

```
@Component({  
  selector: 'app-manager',  
  template: `  
    <button (click)="add()">Add</button>  
    <button (click)="remove()">Remove Last</button>  
    <ng-template #content></ng-template>  
  `,  
})  
export class ManagerComponent {  
  @ViewChild('content', { read: ViewContainerRef })  
  container!: ViewContainerRef;  
  
  components: ComponentRef<any>[] = [];  
  
  add(): void {  
    const ref = this.container.createComponent(BoxComponent);  
    this.components.push(ref);  
  }  
  
  remove(): void {  
    const ref = this.components.pop();  
    ref?.destroy();  
  }  
}
```

7. Practical Examples

7.1 Simple Notification

Concept: A service that shows temporary messages.

```

@Component({
  selector: 'app-notification',
  template: `<div class="note">{{ message }}</div>`,
  styles: [
    .note {
      padding: 12px 16px;
      background: #333;
      color: white;
      border-radius: 4px;
    }
  ]
})
export class NotificationComponent {
  message = '';
}

@Injectable({ providedIn: 'root' })
export class NotificationService {
  private container?: ViewContainerRef;

  setContainer(ref: ViewContainerRef): void {
    this.container = ref;
  }

  show(message: string): void {
    const ref = this.container!.createComponent(NotificationComponent);
    ref.instance.message = message;

    setTimeout(() => ref.destroy(), 3000);
  }
}

```

7.2 Basic Card with Slots

Concept: Simple reusable card with header, body, and footer slots.

```

@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <div class="header">
        <ng-content select="[card-header]"></ng-content>
      </div>

```

```
<div class="body">
  <ng-content></ng-content>
</div>
<div class="footer">
  <ng-content select="[card-footer]"></ng-content>
</div>
</div>
`,
styles: [`
  .card {
    border: 1px solid #ddd;
    border-radius: 8px;
    padding: 16px;
  }
  .header { font-weight: bold; margin-bottom: 12px; }
  .footer { margin-top: 12px; padding-top: 12px; border-top: 1px solid #eee; }
`]
})
export class CardComponent {}
```

Usage:

```
<app-card>
  <h3 card-header>Card Title</h3>
  <p>Card content goes here</p>
  <button card-footer>Action</button>
</app-card>
```

8. Summary and Key Takeaways

8.1 Key Concepts

1. Content Projection (ng-content)

- Allows parent components to pass content to child components
- Creates reusable wrapper components
- Supports single and multi-slot projection
- Uses CSS selectors to target specific content

2. Dynamic Component Loading

- Creates components at runtime programmatically
- Uses `ViewContainerRef.createComponent()`
- Perfect for modals, notifications, and plugins
- `ComponentFactoryResolver` is deprecated (use direct approach)

3. ViewContainerRef

- Container for dynamically created views
- Provides methods to create, insert, move, and remove views
- Works with both components and templates

8.2 Content Projection vs Dynamic Loading

Feature	Content Projection	Dynamic Loading
When	Compile time (in template)	Runtime (programmatic)
Flexibility	Fixed structure	Highly dynamic
Complexity	Simple	More complex
Use Case	Reusable wrappers	Runtime decisions
Performance	Better (static)	Slightly overhead

8.3 Best Practices

Content Projection:

- ☒ Use meaningful selector names (e.g., `[dialog-header]`)
- ☒ Provide a default slot for unmatched content
- ☒ Check for projected content with `@ContentChild`
- ☒ Use `ngProjectAs` when wrapping with structural directives
- ☒ Avoid deep nesting of projections

Dynamic Components:

- ☒ Always destroy ComponentRef when done
- ☒ Store references for cleanup
- ☒ Use ViewContainerRef.clear() to remove all
- ☒ Set inputs immediately after creation
- ☒ Don't forget to unsubscribe from outputs

8.4 Common Use Cases

Use Case	Technique	Example
Reusable Card	Content Projection	Card with header, body, footer slots
Modal Dialog	Dynamic Loading	Open modal on button click
Tabs Component	Content Projection	Tab labels and content slots
Toast Notifications	Dynamic Loading	Show notification service
Layout Wrapper	Content Projection	Header, sidebar, content areas
Widget Dashboard	Dynamic Loading	Add/remove widgets runtime

8.5 Quick Reference

Single Slot Projection:

```

<!-- Child -->
<ng-content></ng-content>

<!-- Parent -->
<app-child>Content here</app-child>

```

Multi-Slot Projection:

```
<!-- Child -->
<ng-content select="[header]"></ng-content>
<ng-content></ng-content>
<ng-content select="[footer]"></ng-content>

<!-- Parent -->
<app-child>
  <div header>Header</div>
  <p>Body</p>
  <div footer>Footer</div>
</app-child>
```

Dynamic Component:

```
// Create
const ref = this.container.createComponent(MyComponent);
ref.instance.property = value;

// Destroy
ref.destroy();

// Clear all
this.container.clear();
```

8.6 Next Steps

- Practice building reusable components with projection
- Implement a modal or toast service with dynamic loading
- Create a dashboard with draggable widgets
- Build a dynamic form generator
- Explore portal API for advanced projections

Additional Resources

- **Content Projection:** <https://angular.io/guide/content-projection>
- **Dynamic Components:** <https://angular.io/guide/dynamic-component-loader>

- **ViewContainerRef:** <https://angular.io/api/core/ViewContainerRef>
- **Portal API:** <https://material.angular.io/cdk/portal/overview>

💡 **Pro Tip:** Use content projection for static, reusable UI patterns and dynamic loading for runtime-determined components. Combine both techniques for maximum flexibility!

